ContentOS — Полная Документация

Версия 1.0 · Апрель 2026

> ***ContentOS** — AI-платформа для продюсеров и экспертов. Берёт на себя 90% операционки: находит клиентов, создаёт контент, ведёт CRM, отправляет рассылки и считает деньги.*

Содержание

1. Обзор проекта
2. Технический стек
3. Установка и запуск
4. Переменные окружения
5. Архитектура
6. Структура файлов
7. Аутентификация и роли
8. Страницы и маршруты
9. API-эндпоинты
10. Библиотеки и утилиты
11. Компоненты
12. Внешние интеграции
13. Хранилище данных
14. AI-агенты
15. Платёжная система
16. Деплой
17. Сбор ошибок

1. Обзор проекта

ContentOS — полноценная SaaS-платформа для цифровых агентств, контент-продюсеров и экспертов. Объединяет CRM, AI-генерацию контента, финансовый учёт, видеоклиппинг и автоматизацию в одном интерфейсе.

Ключевые возможности

Модуль	Описание
Дашборд	KPI в реальном времени, выручка, лиды, AI-дайджест
Кабинет	Личный планировщик задач с планом на 90 дней
Поиск клиентов	AI-парсинг аудитории, рассылки, рекламные объявления
CRM / Лиды	Канбан-воронка от контакта до контракта
Клиенты	Полный профиль, метрики, 9-этапный путь клиента
AI-команда	4 AI-специалиста (продюсер, стратег, копирайтер, таргетолог)
Автоматизация	4-этапный пайплайн при создании клиента

Хаб	Контент-стратегия: 12 вкладок (воронки, боты, лендинги, посевы)
Студия	AI-клиппинг видео (Vizard / Opus), субтитры, публикация
Финансы	Транзакции, статистика доходов/расходов по категориям
Контент	Планировщик публикаций
Вирусный поиск	Тренды YouTube по нишам
Планировщик	Расписание постов
Команда	Управление AI-воркерами, база знаний агентов
Продукты	Управление офферами и продуктами
Пользователи	Управление аккаунтами (только admin)

---

2. Технический стек

Frontend / Framework

Технология	Версия	Назначение
Next.js	15.5.14	App Router, SSR, API routes
React	18.3.1	UI-компоненты
TypeScript	5.4.5	Строгая типизация
Tailwind CSS	3.4.3	Utility-first стили
Lucide React	0.378.0	Иконки
Recharts	2.12.7	Графики и диаграммы
clsx	2.1.1	Условные классы

Backend / Storage

Технология	Назначение
Next.js API Routes	Серверная логика (нет отдельного бэкенда)
Upstash Redis	Основное хранилище (NoSQL / KV)
In-memory fallback	Локальная разработка без Redis
Node.js crypto	JWT + хэширование паролей

Внешние сервисы

Сервис	Назначение
Groq API (llama-3.3-70b)	LLM для AI-агентов и генерации контента

Replicate	Генерация изображений (FLUX) и видео (Minimax)
Vizard.ai	Автоклиппинг видео
Opus Clip	Альтернативный клиппинг
YouTube Data API v3	Тренды и вирусный контент
Google Sheets API v4	Экспорт регистраций
Telegram Bot API	Уведомления администратору
Prodamus	Приём платежей (RU)
Upstash Redis	Облачное KV-хранилище

---

3. Установка и запуск

Требования

- Node.js ≥ 18.x
- npm ≥ 9.x
- Git

Локальная разработка

```
# 1. Клонировать репозиторий
git clone https://github.com/zarinkadesign-pixel/ContentOS
cd ContentOS/frontend

# 2. Установить зависимости
npm install

# 3. Настроить переменные окружения
cp .env.example .env.local
# Заполнить обязательные переменные (см. раздел 4)

# 4. Запустить сервер разработки
npm run dev
# → http://localhost:3000
```

Скрипты

```
npm run dev      # Сервер разработки (Next.js с hot reload)
npm run build    # Production-сборка
```

```
npm run start    # Запуск production-сборки
npm run lint     # ESLint-проверка
```

Быстрая публикация на GitHub

Запустить `publish_to_github.bat` из корня проекта `D:\Content OS\`. Требует установленных `git` и `GitHub CLI (gh)`.

```
Репозиторий: https://github.com/zarinkadesign-pixel/ContentOS
Ветка: main
```

---

4. Переменные окружения

Файл: `frontend/.env.local`

```
# ── AI / LLM ──────────────────────────────────────────────
# Бесплатно: console.groq.com (llama-3.3-70b-versatile)
GROQ_KEY=gsk_...

# ── Генерация медиа ───────────────────────────────────────
# replicate.com — FLUX (изображения), Minimax Video-01 (видео)
REPLICATE_API_TOKEN=r8_...

# ── Аналитика ─────────────────────────────────────────────
# console.cloud.google.com — 10 000 запросов/день бесплатно
YOUTUBE_API_KEY=AIza...

# ── Клиппинг видео ────────────────────────────────────────
OPUS_KEY=...
VIZARD_KEY=...

# ── Redis (хранилище данных) ──────────────────────────────
# console.upstash.com — бесплатный tier
UPSTASH_REDIS_REST_URL=https://...upstash.io
UPSTASH_REDIS_REST_TOKEN=AX...

# ── Аутентификация ────────────────────────────────────────
# Обязательно сменить в production!
JWT_SECRET=contentOS_jwt_secret_dev
ADMIN_EMAIL=admin@amai.media
```

```
ADMIN_PASSWORD=your_strong_password

# ── Telegram-уведомления ──────────────────────────────────
# Создать бота через @BotFather
TELEGRAM_BOT_TOKEN=123456789:AAF...
TELEGRAM_CHAT_ID=-1001234567890   # Отрицательный ID канала/группы

# ── Платежи (Prodamus) ────────────────────────────────────
PRODAMUS_SHOP=your_shop_slug
PRODAMUS_KEY=hmac_secret_key

# ── Google Sheets (экспорт регистраций) ───────────────────
GOOGLE_SHEETS_ID=1BxiMVs0XRA5nFMdKvBdBZjgmUUqptlbs74OgVE2upms
GOOGLE_SA_EMAIL=your-service@project.iam.gserviceaccount.com
GOOGLE_SA_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE
KEY-----\n"
```

Обязательные / Опциональные

Переменная	Обязательна	Без неё
`GROQ_KEY`	✅ Да	AI-агенты не работают
`JWT_SECRET`	✅ Да	Аутентификация сломана
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	✅ Да	Нет входа admin
`UPSTASH_REDIS_REST_URL`	⚠️ Рекомендуется	Fallback в память (данные теряются)
`TELEGRAM_BOT_TOKEN`	⚙️ Опционально	Без уведомлений
`PRODAMUS_*`	⚙️ Опционально	Без платежей
`GOOGLE_SHEETS_ID`	⚙️ Опционально	Без экспорта в таблицы
`REPLICATE_API_TOKEN`	⚙️ Опционально	Без генерации изображений/видео
`YOUTUBE_API_KEY`	⚙️ Опционально	Без вирусного поиска
`OPUS_KEY` / `VIZARD_KEY`	⚙️ Опционально	Без автоклиппинга

---

5. Архитектура

```
┌──────────────────────────────────────────────────────────┐
│                    Next.js 15 App Router                 │
│                                                          │
```

```
│   ┌──────────────┐      ┌──────────────────────┐     │
│   │   Frontend   │      │    API Routes (BE)     │     │
│   │  (app/)      │──────▶│    (app/api/)          │     │
│   │  React 18    │      │    Node.js runtime     │     │
│   └──────────────┘      └──────────┬───────────┘     │
│                                     │                 │
└─────────────────────────────────────┼─────────────────┘
                                      │
            ┌─────────────────────────┼─────────────────┐
            │                         │                 │
      ┌─────▼──────┐           ┌──────▼──────┐   ┌──────▼──────┐
      │  Upstash   │           │   Groq API  │   │  Telegram   │
      │   Redis    │           │ llama-3.3-70b│   │  Bot API    │
      └────────────┘           └─────────────┘   └─────────────┘
            │
      ┌─────▼──────────────────────────────────────────────┐
      │   KV Keys:                                          │
      │   users · clients · leads · finance · settings      │
      │   workspace:tasks · workspace:time                  │
      │   team:tasks · automation:runs                      │
      │   agent:knowledge:{type}                            │
      └────────────────────────────────────────────────────┘
```

Принцип работы

1. **Нет отдельного бэкенда** — всё в Next.js API Routes
2. **Нет SQL** — все данные в Upstash Redis (KV)
3. **In-memory fallback** — работает в dev без Redis (данные теряются при перезапуске)
4. **JWT без библиотек** — кастомная реализация на Node.js crypto
5. **AI через Groq** — функция `callGemini` (исторически) вызывает Groq API

---

6. Структура файлов

```
D:\Content OS\
├── frontend/                        ← Next.js приложение
│   ├── app/
│   │   ├── page.tsx                 ← Лендинг /
│   │   ├── layout.tsx               ← Root layout (BugReporter)
│   │   ├── globals.css              ← Глобальные стили
│   │   ├── (auth)/                  ← Публичные страницы
│   │   │   ├── login/page.tsx       ← Вход
```

```
│   │   │   ├── register/page.tsx    ← Регистрация (платные планы)
│   │   │   └── pricing/page.tsx     ← Тарифы
│   │   ├── (app)/                   ← Защищённые страницы
│   │   │   ├── dashboard/page.tsx
│   │   │   ├── workspace/page.tsx   ← Кабинет + план 90 дней
│   │   │   ├── hub/page.tsx
│   │   │   ├── studio/page.tsx
│   │   │   ├── clients/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   ├── crm/page.tsx
│   │   │   ├── finance/page.tsx
│   │   │   ├── content/page.tsx
│   │   │   ├── team/page.tsx
│   │   │   ├── automation/page.tsx
│   │   │   ├── scheduler/page.tsx
│   │   │   ├── products/page.tsx
│   │   │   ├── users/page.tsx
│   │   │   ├── prospecting/page.tsx
│   │   │   ├── viral-finder/page.tsx
│   │   │   ├── funnels/page.tsx
│   │   │   ├── campaigns/page.tsx
│   │   │   └── generate/page.tsx
│   │   └── api/
│   │       ├── auth/
│   │       │   ├── login/route.ts
│   │       │   ├── register/route.ts
│   │       │   ├── free/route.ts
│   │       │   ├── demo/route.ts
│   │       │   ├── logout/route.ts
│   │       │   └── me/route.ts
│   │       ├── clients/
│   │       │   ├── route.ts
│   │       │   └── [id]/route.ts
│   │       ├── leads/
│   │       │   ├── route.ts
│   │       │   └── [id]/route.ts
│   │       ├── workspace/
│   │       │   ├── tasks/route.ts
│   │       │   ├── tasks/[id]/route.ts
│   │       │   ├── time/route.ts
│   │       │   └── stats/route.ts
```

```
│   │       ├── team/
│   │       │   ├── tasks/route.ts
│   │       │   ├── tasks/[id]/route.ts
│   │       │   └── knowledge/route.ts
│   │       ├── automation/
│   │       │   ├── route.ts
│   │       │   └── [id]/route.ts
│   │       ├── finance/route.ts
│   │       ├── dashboard/route.ts
│   │       ├── hub/route.ts
│   │       ├── studio/route.ts
│   │       ├── agents/route.ts
│   │       ├── bugs/route.ts
│   │       ├── settings/route.ts
│   │       ├── payments/webhook/route.ts
│   │       ├── generate/
│   │       │   ├── image/route.ts
│   │       │   └── video/route.ts
│   │       ├── vizard/route.ts
│   │       ├── opus/route.ts
│   │       └── trends/route.ts
│   ├── components/
│   │   ├── Sidebar.tsx
│   │   ├── KPICard.tsx
│   │   ├── KanbanBoard.tsx
│   │   ├── SavePanel.tsx
│   │   └── BugReporter.tsx
│   ├── lib/
│   │   ├── auth.ts          ← JWT + хэширование
│   │   ├── kv.ts            ← Redis / in-memory storage
│   │   ├── gemini.ts        ← Groq API client (callGemini)
│   │   ├── agents.ts        ← Системные промпты AI-агентов
│   │   ├── automation.ts    ← Пайплайн автоматизации
│   │   ├── telegram.ts      ← Telegram-уведомления
│   │   ├── sheets.ts        ← Google Sheets экспорт
│   │   ├── prodamus.ts      ← Генерация платёжных ссылок
│   │   ├── vizard.ts        ← Vizard.ai клиент
│   │   ├── opus.ts          ← Opus Clip клиент
│   │   ├── types.ts         ← TypeScript интерфейсы
│   │   ├── api.ts           ← Frontend API-клиент
│   │   └── demo-guard.ts    ← Защита demo-режима
│   ├── middleware.ts        ← Auth + role-based routing
```

```
│   ├── tailwind.config.ts
│   ├── next.config.ts
│   └── package.json
├── docs/
│   ├── ContentOS_Documentation_v1.0.md  ← Этот файл
│   └── ContentOS_Documentation_v1.0.pdf
├── publish_to_github.bat    ← Авто-публикация на GitHub
└── .gitignore
```

---

7. Аутентификация и роли

Роли пользователей

Роль	Доступ	Описание
`admin`	Всё	Полный доступ, все страницы
`hub`	Hub + базовые	Платная подписка Hub ($29/мес или $290/год)
`studio`	Studio + базовые	Платная подписка Studio ($15/мес или $150/год)
`free`	Ограниченный	Бесплатная регистрация, нет /users
`demo`	Только чтение	Seed-данные, без сохранения

Токен JWT

Алгоритм: `HS256` (HMAC-SHA256) Хранение: HttpOnly cookie `contentOS_token` Срок: 30 дней

```typescript
interface TokenPayload {
  sub: string;           // user ID
  email: string;
  name: string;
  role: "admin" | "hub" | "studio" | "demo" | "free";
  plan: string | null;
  plan_expires_at: string | null;
  iat: number;
  exp: number;
}
```

Поток аутентификации

```
Пользователь → /login
    │
    ├── Admin (ADMIN_EMAIL + ADMIN_PASSWORD из .env)
    │     └── Мгновенный JWT → /dashboard
    │
    ├── Обычный пользователь
    │     ├── Найти в Redis по email
    │     ├── Проверить хэш пароля (HMAC-SHA256)
    │     ├── Проверить plan_active
    │     └── JWT → /dashboard
    │
    └── Бесплатный /api/auth/free
          ├── Создать user с role="free"
          ├── Записать в Redis
          ├── Google Sheets → лист "Бета-тестеры"
          ├── Telegram-уведомление admin
          └── JWT → /dashboard
```

Платная регистрация

```
/register → /api/auth/register
    ├── Создать пользователя (plan_active=false)
    ├── Google Sheets → лист "Регистрации"
    ├── Telegram-уведомление admin
    ├── Prodamus payment link
    └── Редирект на оплату

/api/payments/webhook (Prodamus callback)
    ├── Проверить HMAC-подпись
    ├── Найти пользователя по orderId
    ├── plan_active=true, plan_expires_at=now+30d/365d
    └── Пользователь может войти
```

Middleware

Файл: `frontend/middleware.ts`

Публичные пути (без авторизации):

- `/` , `/login` , `/register` , `/pricing`
- `/api/auth/*` , `/api/payments/*`

Все остальные пути требуют валидный JWT.

8. Страницы и маршруты

Публичные `/(auth)`

`/` — Лендинг

Маркетинговая страница. Интерактивные мокапы всех модулей. Кнопки: «Начать бесплатно» (free), «Студия $15/мес» (платно). Анимированный демо-превью.

`/login` — Вход

Email + пароль. Ссылка на бесплатную регистрацию. Редирект после успешного входа по роли.

`/register` — Регистрация (платные планы)

Форма: имя, email, пароль, выбор плана (Hub/Studio × ежемесячно/ежегодно). После регистрации — редирект на Prodamus-оплату.

`/pricing` — Тарифы

Сравнение планов. Кнопки «Начать».

Защищённые `/(app)`

`/dashboard` — Дашборд

- **KPI-карточки**: Выручка, Прогресс к цели %, Клиентов, Лидов
- **График**: Ежемесячная выручка (Recharts BarChart)
- **Воронка лидов**: визуальный разброс по стадиям
- **Статус автоматизации**: активные воркеры
- **AI-дайджест**: анализ недели от Groq
- **Редактирование цели**: inline-поле для monthly_target
- **Авто-обновление**: каждые 60 секунд

`/workspace` — Кабинет (только admin)

- **Планировщик задач**: вкладки Сегодня / Неделя / Месяц
- **AI-генерация задач**: авто-создание задач по контексту
- **Таймер**: трекинг рабочего времени по категориям
- **Статистика недели**: часов отработано, задач выполнено
- **AI-выполнение задач**: нажать «AI» → агент выполняет задачу
- **План 90 дней** (только admin): 3 фазы × 4 недели, статусы ✅/📅/AI, localStorage

`/prospecting` — Поиск клиентов

- **Парсинг аудитории**: выбор платформы + ниши → генерация лидов
- **AI-сообщения**: персональный outreach для каждого лида
- **Массовая рассылка**: AI-шаблоны по нише и цели
- **Таргетированная реклама**: объявления для Instagram/Facebook/Threads/YouTube
- **Воронка лидов**: статусы (новый → написали → ответили → годный → закрыт)

`/crm` — CRM / Лиды

Полный канбан продаж с 6 стадиями. Создание, редактирование, перемещение лидов.

`/clients` — Клиенты

- Список клиентов с метриками
- Фильтр по статусу
- Создание нового клиента (→ запускает automation pipeline)

`/clients/[id]` — Профиль клиента

- Полная карточка: соцсети, аудитория, доход, нишевые метрики
- 9-этапный путь клиента с прогрессом
- Чеклист готовности
- Анализ через AI
- Продуктовая линейка
- Контент-план

`/team` — AI-команда

- **4 AI-воркера**: Продюсер, Стратег, Копирайтер, Meta-таргетолог
- Назначение задач на воркера + клиента
- История выполненных задач с результатами
- **База знаний**: кастомные инструкции для каждого агента
- Выполнение через Groq API с полным контекстом клиента

`/automation` — Автоматизация

- Список запусков пайплайна (producer → strategist → copywriter → metaads)
- Статусы шагов: pending / running / done / error
- Создание ручного запуска для клиента
- Streaming результатов (SSE)

`/hub` — Контент-Хаб (tier: hub)

12-вкладочный стратегический центр:

Вкладка	Описание
Главная	Профиль пользователя + быстрые метрики
Стратег	AI-стратегия под нишу и цель
Карта воронки	Визуальная воронка с конверсиями
Текст	Генерация текстов (посты, сторис, Reels)
Реклама	Рекламные объявления по платформам
Бот	Цепочки сообщений (прогрев, продажи, онбординг)
Продукты	Линейка продуктов и офферов
Цели	Финансовые цели и KPI
Недельный план	AI-план на неделю
Визуал	Идеи для визуала и шаблоны
Лендинги	Структура лендингов

Заметки	Личные заметки

`/studio` — Студия (tier: studio)

- Загрузка URL видео (YouTube / Loom / прямые ссылки)
- Автоклиппинг через Vizard или Opus
- Выбор соотношения сторон (9:16, 1:1, 16:9)
- Длина клипов (30/60/90 сек)
- Поллинг статуса → список готовых клипов
- AI-подписи и хэштеги
- Оценка вирусности (0–100)
- Планирование публикации

`/finance` — Финансы

- Список транзакций (доход/расход)
- Добавление транзакции с категорией
- Ежемесячный график выручки
- Разбивка расходов (SaaS-инструменты)
- Итоги за период

`/content` — Контент

Планировщик публикаций по дням недели.

`/scheduler` — Планировщик

Расписание постов для соцсетей.

`/products` — Продукты

Управление офферами: мини-продукты, наставничество, продюсирование. Цены, описания, количество продаж.

`/viral-finder` — Вирусный поиск

Тренды YouTube по категориям. Анализ вирусного контента в нише.

`/users` — Пользователи (только admin)

Список всех аккаунтов, роли, статус подписки, дата регистрации.

---

9. API-эндпоинты

Аутентификация

Метод	Путь	Описание
POST	`/api/auth/login`	Вход по email/паролю
POST	`/api/auth/register`	Регистрация (платный план)
POST	`/api/auth/free`	Бесплатная регистрация
POST	`/api/auth/demo`	Демо-доступ

POST	`/api/auth/logout`	Выход (удалить cookie)
GET	`/api/auth/me`	Текущий пользователь

Клиенты

Метод	Путь	Описание
GET	`/api/clients`	Список клиентов
POST	`/api/clients`	Создать клиента (+ запуск automation)
GET	`/api/clients/[id]`	Профиль клиента
PUT	`/api/clients/[id]`	Обновить клиента
DELETE	`/api/clients/[id]`	Удалить клиента
POST	`/api/clients/[id]/analyze`	AI-анализ метрик

Лиды

Метод	Путь	Описание
GET	`/api/leads`	Список лидов
POST	`/api/leads`	Создать лид
PUT	`/api/leads/[id]`	Обновить лид
PUT	`/api/leads/[id]/stage`	Сменить стадию
DELETE	`/api/leads/[id]`	Удалить лид

Кабинет (Workspace)

Метод	Путь	Описание
GET	`/api/workspace/tasks`	Задачи (фильтр: today/week/month)
POST	`/api/workspace/tasks`	Создать задачу
PUT	`/api/workspace/tasks/[id]`	Обновить задачу
POST	`/api/workspace/tasks/[id]/start`	Выполнить задачу через AI
DELETE	`/api/workspace/tasks/[id]`	Удалить задачу
GET	`/api/workspace/time`	Сессии трекинга времени
POST	`/api/workspace/time`	Старт/стоп таймера
GET	`/api/workspace/stats`	Статистика продуктивности

AI-команда

Метод	Путь	Описание
GET	`/api/team/tasks`	Все задания агентов
POST	`/api/team/tasks`	Новое задание (worker_id + client_id)
DELETE	`/api/team/tasks/[id]`	Удалить задание
GET	`/api/team/knowledge`	База знаний агента
POST	`/api/team/knowledge`	Добавить знание
DELETE	`/api/team/knowledge/[id]`	Удалить знание

Автоматизация

Метод	Путь	Описание
GET	`/api/automation`	Все запуски пайплайна
POST	`/api/automation`	Запустить пайплайн для клиента
GET	`/api/automation/[id]`	Детали запуска
POST	`/api/automation/[id]/stream`	Стриминг шага (SSE)

Контент и Хаб

Метод	Путь	Action	Описание
POST	`/api/hub`	`weeklyDigest`	AI-дайджест недели
POST	`/api/hub`	`autoPilot`	Авто-стратегия
POST	`/api/hub`	`strategy`	Контент-стратегия
POST	`/api/hub`	`socialAnalysis`	Анализ соцсети
POST	`/api/hub`	`content`	Генерация поста
POST	`/api/hub`	`marketAnalysis`	Анализ рынка
POST	`/api/hub`	`competitorAnalysis`	Анализ конкурентов
POST	`/api/hub`	`launchPlan`	План запуска
POST	`/api/hub`	`outreachMessage`	Персональное сообщение
POST	`/api/hub`	`bulkOutreachScript`	Шаблоны для массовой рассылки
POST	`/api/hub`	`adCopy`	Рекламное объявление
POST	`/api/hub`	`advise`	AI-совет по задаче (план 90 дней)

Студия / Видео

Метод	Путь	Описание

POST	`/api/studio`	AI-копирайтинг (Instagram/TG/TikTok/YouTube/Email/Blog)
POST	`/api/vizard/clip`	Отправить видео в Vizard
GET	`/api/vizard/project/[id]`	Статус клиппинга
POST	`/api/vizard/social`	AI-подписи Vizard
POST	`/api/vizard/publish`	Опубликовать клип
POST	`/api/opus/clip`	Отправить видео в Opus
GET	`/api/opus/project/[id]`	Статус Opus
POST	`/api/generate/image`	Сгенерировать изображение (FLUX)
POST	`/api/generate/video`	Сгенерировать видео (Minimax)

Прочее

Метод	Путь	Описание
GET	`/api/dashboard`	KPI-агрегация для дашборда
GET	`/api/finance`	Транзакции и статистика
POST	`/api/finance/transactions`	Добавить транзакцию
GET	`/api/settings`	Настройки пользователя
PUT	`/api/settings`	Обновить настройки
GET	`/api/trends`	Тренды YouTube
POST	`/api/payments/webhook`	Webhook Prodamus (активация плана)
POST	`/api/bugs`	Отправить ошибку (bug report)

---

10. Библиотеки и утилиты

`lib/auth.ts`

Кастомная JWT-реализация без внешних зависимостей.

```
// Создать токен
createToken(payload: Omit<TokenPayload, 'iat' | 'exp'>): string

// Верифицировать токен
verifyToken(token: string): TokenPayload | null

// Хэш пароля
```

```
hashPassword(password: string, salt?: string): { hash: string; salt:
string }

// Проверить пароль
verifyPassword(password: string, hash: string, salt: string): boolean

// Хелперы ролей
canAccessHub(payload): boolean   // admin | hub (активный)
canAccessStudio(payload): boolean // admin | studio (активный)
isAdmin(payload): boolean
isDemo(payload): boolean
isFree(payload): boolean
```

`lib/kv.ts`

Обёртка над Upstash Redis с in-memory fallback.

KV-ключи:

Ключ	Тип	Описание
`clients`	`Client[]`	Все клиенты
`leads`	`Lead[]`	Все лиды
`users`	`User[]`	Все пользователи
`finance`	`Finance`	Транзакции и настройки
`settings:{userId}`	`Settings`	Настройки пользователя
`workspace:tasks:{userId}`	`Task[]`	Задачи пользователя
`workspace:time:{userId}`	`TimeSession[]`	Сессии трекинга
`team:tasks`	`TeamTask[]`	Задания AI-агентов
`automation:runs`	`AutomationRun[]`	Запуски пайплайна
`agent:knowledge:{type}`	`KnowledgeItem[]`	База знаний агента

`lib/gemini.ts`

Groq API клиент (название сохранено из исторических соображений).

```
// Основной вызов LLM
callGemini(
  systemPrompt: string,
  userPrompt: string,
```

```
  maxTokens: number = 3000  // free: 1000, paid: 3000
): Promise<string>

// Построить контекст клиента для промптов
buildClientContext(client: Client): string
```

Модель: `llama-3.3-70b-versatile` (Groq)

`lib/agents.ts`

Системные промпты для 4 AI-специалистов с расширенной базой знаний:

- Формулы копирайтинга (AIDA, PAS, BAB, PASTOR, 4U, StoryBrand)
- Структура коротких видео (хук 0–3с, build 3–15с, payoff + CTA)
- Bowtie-воронка (retention/expansion/advocacy)
- Content-to-funnel mapping (TOFU/MOFU/BOFU)
- 6-этапный фреймворк роста
- Multi-agent Marketing Crew паттерн
- Автоматизация и делегирование

`lib/telegram.ts`

Уведомления в Telegram-канал/группу администратора.

```
notifyNewFreeUser(data: { name, email, createdAt }): Promise<void>
notifyNewBetaUser(data: { name, email, createdAt }): Promise<void>
notifyNewPaidUser(data: { name, email, plan, createdAt }): Promise<void>
notifyBug(data: { message, url, userAgent, role, timestamp }):
Promise<void>
```

Все функции fire-and-forget (не блокируют ответ).

`lib/sheets.ts`

Экспорт в Google Sheets через Service Account.

```
// Лист "Регистрации"
appendUserToSheet(row: { date, name, email, plan, status }): Promise<void>

// Лист "Бета-тестеры"
appendBetaUserToSheet(row: { date, name, email, source }): Promise<void>
```

`lib/prodamus.ts`

Генерация платёжных ссылок с HMAC-SHA256 подписью.

Планы:

Ключ	Название	Цена
`hub_monthly`	Хаб — ежемесячно	$29
`hub_yearly`	Хаб — ежегодно	$290
`studio_monthly`	Студия — ежемесячно	$15
`studio_yearly`	Студия — ежегодно	$150

---

11. Компоненты

`Sidebar.tsx`

Основная навигация. Секции:

ОБЗОР: Дашборд, Кабинет **ПРИВЛЕЧЕНИЕ**: Поиск клиентов, CRM/Лиды, Тренды **РАБОТА**: Клиенты, Команда, Автоматизация, Контент, Планировщик **МОНЕТИЗАЦИЯ**: Финансы, Продукты, Кампании, Воронки **ИНСТРУМЕНТЫ**: Хаб, Студия, Генератор, Пользователи

`BugReporter.tsx`

Невидимый клиентский компонент в `app/layout.tsx`.

```
// Перехватывает все JS-ошибки
window.addEventListener("error", onError)
window.addEventListener("unhandledrejection", onUnhandledRejection)

// Отправляет на /api/bugs
fetch("/api/bugs", {
  method: "POST",
  body: JSON.stringify({ message, stack, url, userAgent }),
  keepalive: true  // работает даже при закрытии страницы
})
```

Фильтрует: ResizeObserver, ChunkLoadError (шум).

`KPICard.tsx`

Карточка метрики с иконкой, значением и изменением в %.

`KanbanBoard.tsx`

Перетаскиваемая канбан-доска для лидов и задач.

`SavePanel.tsx`

Плашка состояния сохранения (saved / saving / error).

---

12. Внешние интеграции

Groq API

- **URL**: `https://api.groq.com/openai/v1/chat/completions`
- **Модель**: `llama-3.3-70b-versatile`
- **Токены**: 1 000 (free/demo) / 3 000 (paid)
- **Бесплатный tier**: console.groq.com

Google Sheets API v4

- Аутентификация через Service Account (JWT)
- Две вкладки: `Регистрации` и `Бета-тестеры`
- Fire-and-forget (не блокирует регистрацию при ошибке)

Telegram Bot API

- Все уведомления через `sendMessage` с `parse_mode: HTML`
- Срабатывает на: free-регистрацию, beta-регистрацию, платную регистрацию, JS-ошибки

Prodamus

- Платёжный шлюз (RU)
- HMAC-SHA256 подпись запроса
- Webhook на `/api/payments/webhook` для активации плана

Upstash Redis

- Основное KV-хранилище
- REST API (HTTP-запросы)
- Fallback: in-memory Map при отсутствии credentials

Vizard.ai / Opus Clip

- Сабмит URL видео → polling → список клипов
- Оценка вирусности
- AI-подписи

Replicate

- **FLUX Schnell**: генерация изображений
- **Minimax Video-01**: генерация коротких видео

YouTube Data API v3

- Поиск трендов по категориям

- 10 000 единиц/день бесплатно

---

13. Хранилище данных

Все данные хранятся в **Upstash Redis** (NoSQL KV). В dev-режиме без Redis — in-memory Map (данные теряются при перезапуске).

Основные модели

User

```
{
  id: string              // "user_abc123"
  email: string
  name: string
  password_hash: string
  password_salt: string
  role: "admin" | "hub" | "studio" | "free"
  plan: string | null
  plan_expires_at: string | null
  plan_active: boolean
  created_at: string      // ISO datetime
  last_login: string | null
}
```

Client

```
{
  id: string
  name: string
  niche: string
  contact: string         // @handle
  followers: number
  reach: number
  engagement: number
  income: number
  status: "active" | "paused" | "done"
  journey_stage: number   // 0-8
  products: Product[]
  content_plan: ContentPlan
```

```
  created_at: string
}
```

Lead

```
{
  id: string
  name: string
  source: string
  niche: string
  product: string
  contact: string
  notes: string
  stage: "new" | "contacted" | "replied" | "interested" | "call" |
"contract"
  created_at: string
}
```

Task (Workspace)

```
{
  id: string
  title: string
  description: string
  period: "today" | "week" | "month"
  status: "todo" | "in_progress" | "done"
  category: string
  due_date: string
  ai_result?: string
  ai_worker?: string
  ai_status?: "pending" | "running" | "done" | "error"
  created_at: string
}
```

---

14. AI-агенты

4 специалиста

Агент	Роль	Специализация
`producer`	Продюсер	Стратегия, запуски, монетизация
`strategist`	Стратег	Контент-стратегия, позиционирование
`copywriter`	Копирайтер	Тексты, скрипты, формулы продаж
`metaads`	Meta-таргетолог	Реклама Facebook/Instagram

Архитектура агента

```
Запрос задачи → buildClientContext(client)
    ↓
Системный промпт агента (lib/agents.ts)
    + База знаний агента (Redis: agent:knowledge:{type})
    + Контекст клиента (метрики, ниша, продукты, цели)
    ↓
callGemini(system, prompt, maxTokens)
    ↓
Groq llama-3.3-70b-versatile
    ↓
Результат → сохранить в Redis + вернуть пользователю
```

Контекст клиента включает

- Имя, ниша, контакт
- Аудитория (подписчики, охват, вовлечённость)
- Доход и финансовые цели
- Этап пути клиента
- Продуктовая линейка
- Стратегия позиционирования

База знаний

Каждый агент имеет персональную базу знаний (до N записей):

- Добавляется через UI `/team`
- Хранится в Redis: `agent:knowledge:{agentType}`
- Добавляется в системный промпт при каждом вызове

---

15. Платёжная система

Тарифы

Тариф	Ежемесячно	Ежегодно	Доступ
Studio	$15/мес	$150/год	/studio + видео
Hub	$29/мес	$290/год	/hub + стратегия

Процесс оплаты

```
1. /register → выбор плана
2. POST /api/auth/register
   → создать user (plan_active=false)
   → создать orderId: "{plan}_{userId}_{timestamp}"
   → generateProdamusLink({ email, plan, orderId })
3. Редирект на Prodamus checkout
4. Пользователь оплачивает
5. POST /api/payments/webhook (от Prodamus)
   → проверить HMAC-подпись
   → найти user по orderId
   → plan_active=true
   → plan_expires_at = now + 30/365 дней
6. Пользователь входит в систему
```

---

16. Деплой

Vercel (рекомендуется)

```
# 1. Установить Vercel CLI
npm install -g vercel

# 2. Деплой
cd frontend
vercel --prod

# 3. Добавить переменные окружения в Vercel Dashboard
#    Settings → Environment Variables → добавить все из .env.local
```

Upstash Redis: Vercel Marketplace → Upstash → Create Database → скопировать URL и Token.

Railway

```
# Из корня D:\Content OS\
railway login
railway init
railway up
# Добавить все env vars в Railway Dashboard
```

Docker (опционально)

```
FROM node:18-alpine
WORKDIR /app
COPY frontend/package*.json ./
RUN npm ci --only=production
COPY frontend/ .
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

Требования к хостингу

- Node.js 18+
- Поддержка Next.js 15 (edge runtime)
- HTTPS (для HttpOnly cookies)
- Webhook URL для Prodamus (публичный HTTPS)

---

17. Сбор ошибок

Клиентская сторона — `BugReporter.tsx`

Глобальный перехват в `app/layout.tsx`:

```
window.addEventListener("error", onError)           // JS-ошибки
window.addEventListener("unhandledrejection", onRejection) // Promise-
ошибки
// → POST /api/bugs { message, stack, url, userAgent }
```

Серверная сторона — `/api/bugs/route.ts`

```
// Запись в файл
const LOG_FILE = path.resolve(process.cwd(), "bugs.log")
fs.appendFileSync(LOG_FILE, JSON.stringify(entry) + "\n")

// Telegram-уведомление
notifyBug({ message, url, userAgent, role, timestamp })
```

Формат записи

```
{
  "timestamp": "2026-04-02T10:00:00.000Z",
  "role": "admin",
  "url": "http://localhost:3000/workspace",
  "message": "TypeError: Cannot read...",
  "stack": "at Component (page.tsx:42:5)...",
  "userAgent": "Mozilla/5.0..."
}
```

Файл `bugs.log` создаётся в корне `frontend/` и **не коммитится** (в `.gitignore`).

Changelog

v1.0 — Апрель 2026

Новые функции:

- ✅ План на 90 дней в Кабинете (фазы апрель/май/июнь, реальный план)
- ✅ Анализ соцсетей перед генерацией стратегии
- ✅ Сбор ошибок: BugReporter → bugs.log + Telegram
- ✅ Google Sheets: лист «Бета-тестеры» для free-регистраций
- ✅ Telegram-уведомления на все типы регистраций
- ✅ Ограничение токенов: free = 1 000, paid = 3 000
- ✅ Таргетированная реклама в модуле поиска клиентов
- ✅ Расширенная база знаний AI-агентов

Исправления:

- ✅ SWC parse error в `prospecting/page.tsx` (stale HMR cache)
- ✅ DefaultBotProperties import в модуле телеграм-бота

ContentOS v1.0 · Copyright © 2026 AMAImedia.com · Все права защищены